
Problem 8.15. Consider the following two-person version of the language PUZZLE that was described in Problem 7.28. Each player starts with an ordered stack of puzzle cards. The players take turns placing the cards in order in the box and may choose which side faces up. Player I wins if all hole positions are blocked in the final stack, and Player II wins if some hole position remains unblocked. Show that the problem of determining which player has a winning strategy for a given starting configuration of the cards is PSPACE-complete.

Proof. Let 2-$PUZZLE = \{\langle C_1, C_2, S, p\rangle \mid C_1,\ C_2$ and S are ordered stacks of cards, representing current configuration of the cards of Player 1, Player 2 and stack, such that Player 1 has a winning strategy if player p takes the first turn$\}$. To show that 2-$PUZZLE$ is PSPACE-complete, we must show that it is in PSPACE and that all PSPACE problems are polynomial time reducible to it. To prove 2-$PUZZLE \in$ PSPACE, we give a polynomial space algorithm M.

$M =$ "On input $\langle C_1, C_2, S, p\rangle$, where C_1, C_2 and S are ordered stacks of cards, and p is an integer 1 or 2:

1. If both C_1 and C_2 are empty, then:
2. If all hole positions are blocked in stack S, then *accept*. Otherwise, *reject*.
3. If $p = 1$, then:
4. If stack C_1 is not empty, then:
5. Remove top most card from C_1, and place it on top of S.
6. Recursively call M on $\langle C_1, C_2, S, 2\rangle$.
7. Remove top most card from S, flip it and place it back on top of S.
8. Recursively call M on $\langle C_1, C_2, S, 2\rangle$.
9. Remove top most card from S, flip it and place it on top of C_1.
10. If one of the recursive calls accept, *accept*. Otherwise, *reject*.
11. Otherwise, recursively call M on $\langle C_1, C_2, S, 2\rangle$, and operate as M afterwards.
12. If $p = 2$, then:
13. If stack C_2 is not empty, then:
14. Remove top most card from C_2, and place it on top of S.
15. Recursively call M on $\langle C_1, C_2, S, 1\rangle$.
16. Remove top most card from S, flip it and place it back on top of S.

17. Recursively call M on $\langle C_1, C_2, S, 1\rangle$.

18. Remove top most card from S, flip it and place it on top of C_2.

19. If both of the recursive calls accept, *accept*. Otherwise, *reject*.

20. Otherwise, recursively call M on $\langle C_1, C_2, S, 1\rangle$, and operate as M afterwards."

□